

```

❷ def reset(self): # called when starting a new game
    self.baddiesList = []
    self.nFramesTilNextBaddie = BaddieMgr.ADD_NEW_BADDIE_RATE

❸ def update(self):
    # Tell each Baddie to update itself
    # Count how many Baddies have fallen off the bottom
    nBaddiesRemoved = 0
    ❹ baddiesListCopy = self.baddiesList.copy()
    for oBaddie in baddiesListCopy:
        ❺ deleteMe = oBaddie.update()
        if deleteMe:
            self.baddiesList.remove(oBaddie)
            nBaddiesRemoved = nBaddiesRemoved + 1

    # Check if it's time to add a new Baddie
    ❻ self.nFramesTilNextBaddie = self.nFramesTilNextBaddie - 1
    if self.nFramesTilNextBaddie == 0:
        oBaddie = Baddie(self.window)
        self.baddiesList.append(oBaddie)
        self.nFramesTilNextBaddie = BaddieMgr.ADD_NEW_BADDIE_RATE

    # Return the count of Baddies that were removed
    return nBaddiesRemoved

❽ def draw(self):
    for oBaddie in self.baddiesList:
        oBaddie.draw()

❽ def hasPlayerHitBaddie(self, playerRect):
    for oBaddie in self.baddiesList:
        if oBaddie.collide(playerRect):
            return True
    return False

```

Listing 16-9: The `BaddieMgr` class

The `__init__()` method calls ❶ the `BaddieMgr`'s own `reset()` method to set the list of `Baddie` objects to the empty list. We use the frame-counting approach to create a new `Baddie` relatively often, to keep things interesting. We use the instance variable `self.nFramesTilNextBaddie` for counting frames.